Problem 01 · Real and Imaginary Roots of a Quadratic Equation ($ax^2 + bx + c = 0$)

$D = b^2 - 4ac$; $D > 0$: $x = (-b \pm \sqrt{D}) / 2a$; $D = 0$: $x = -b / 2a$; $D < 0$: real $= -b / 2a$, imag $= \sqrt{(-D)} / 2a$

Algorithm

1. Start. Read the coefficients a, b, c.
2. If a == 0, display "Invalid input" and stop.
3. Calculate D = (b·b) − (4·a·c).
4. If D > 0: $x_1 = (-b + \sqrt{D})/(2a)$, $x_2 = (-b - \sqrt{D})/(2a)$; display real and distinct roots.
5. Else if D == 0: $x_1 = x_2 = -b/(2a)$; display real and equal roots.
6. Else: realPart = −b/(2a), imagPart = √(−D)/(2a); display realPart ± imagPart·i.
7. Stop.

C Code

```c
#include <stdio.h>
#include <math.h>

int main() {
  double a, b, c;
  double discriminant, root1, root2, realPart, imagPart;
  // Input coefficients
  printf("Enter coefficients a, b and c: ");
  scanf("%lf %lf %lf", &a, &b, &c);
  // Check if valid quadratic equation
  if (a == 0) {
    printf("Invalid input! 'a' cannot be 0 "
           "in a quadratic equation.\n");
    return 0;
  }
  // Calculate discriminant
  discriminant = (b * b) - (4 * a * c);
  // Case 1: Real and Distinct roots
  if (discriminant > 0) {
    root1 = (-b + sqrt(discriminant)) / (2 * a);
    root2 = (-b - sqrt(discriminant)) / (2 * a);
    printf("\nRoots are real and distinct.\n");
    printf("Root 1 = %.2lf\n", root1);
    printf("Root 2 = %.2lf\n", root2);
  }
  // Case 2: Real and Equal roots
  else if (discriminant == 0) {
    root1 = root2 = -b / (2 * a);
    printf("\nRoots are real and equal.\n");
    printf("Root 1 = Root 2 = %.2lf\n", root1);
  }
  // Case 3: Complex and Imaginary roots
  else {
    realPart = -b / (2 * a);
    imagPart = sqrt(-discriminant) / (2 * a);
    printf("\nRoots are complex and imaginary.\n");
    printf("Root 1 = %.2lf + %.2lfi\n", realPart, imagPart);
    printf("Root 2 = %.2lf - %.2lfi\n", realPart, imagPart);
  }
```

```
  return 0;
}
```

Sample Input / Output (input is typed in blue)

```
#1 Enter coefficients a, b and c: 1 -5 6

   Roots are real and distinct.
   Root 1 = 3.00
   Root 2 = 2.00
```

```
#2 Enter coefficients a, b and c: 1 -2 1

   Roots are real and equal.
   Root 1 = Root 2 = 1.00
```

```
#3 Enter coefficients a, b and c: 1 2 5

   Roots are complex and imaginary.
   Root 1 = -1.00 + 2.00i
   Root 2 = -1.00 - 2.00i
```

Problem 02 · Factorial of a Positive Integer

n! = n × (n − 1) × (n − 2) × … × 2 × 1; 0! = 1, 1! = 1; undefined for negative n (use unsigned long long)

Algorithm

1. Start. Read an integer n.
2. If n < 0, display “Factorial of a negative number does not exist” and stop.
3. Initialize factorial = 1.
4. Loop i = 1 to n: factorial = factorial × i.
5. Display factorial. Stop.

C Code

```c
#include <stdio.h>

int main() {
  int n;
  unsigned long long factorial = 1;
  // Input positive integer
  printf("Enter a positive integer: ");
  scanf("%d", &n);
  // Check if the input is negative
  if (n < 0) {
    printf("Error! Factorial of a negative number "
           "doesn't exist.\n");
  } else {
    // Calculate factorial using loop
    for (int i = 1; i <= n; i++) {
      factorial *= i;
    }
    printf("Factorial of %d = %llu\n", n, factorial);
  }
  return 0;
}
```

Sample Input / Output (input is typed in blue)

#1
```
Enter a positive integer: 5
Factorial of 5 = 120
```

#2
```
Enter a positive integer: 10
Factorial of 10 = 3628800
```

#3
```
Enter a positive integer: -3
Error! Factorial of a negative number doesn't exist.
```

Problem 03 · Pyramid of Digits Using Nested Loops

Row i (1 ≤ i ≤ N): (N − i) spaces, digits (i + j) mod 10 for j = 0…i−1, then for j = i−2 down to 0

Algorithm

1. Start. Set the number of rows N = 8.
2. For i = 1 to N:
   a. Space loop: for space = 1 to N − i, print " ".
   b. Ascending loop: for j = 0 to i − 1, print (i + j) mod 10.
   c. Descending loop: for j = i − 2 down to 0, print (i + j) mod 10.
   d. Print a newline.
3. Stop.

C Code

```c
#include <stdio.h>

int main() {
  int N = 8; // Number of rows in the pyramid
  for (int i = 1; i <= N; i++) {
    // 1. Print leading spaces for alignment
    for (int space = 1; space <= N - i; space++) {
      printf(" ");
    }
    // 2. Print ascending digit sequence
    for (int j = 0; j < i; j++) {
      printf("%d", (i + j) % 10);
    }
    // 3. Print descending digit sequence
    for (int j = i - 2; j >= 0; j--) {
      printf("%d", (i + j) % 10);
    }
    // Move to the next line
    printf("\n");
  }
  return 0;
}
```

Sample Input / Output (value of N is set in the code)

#1	N = 4 1 232 34543 4567654	#2	N = 6 1 232 34543 4567654 567898765 67890109876	#3	N = 8 1 232 34543 4567654 567898765 67890109876 7890123210987 890123454321098

Problem 04 · Decimal to Binary, Octal and Hexadecimal

Successive division by base b (2, 8, 16): remainder = n mod b, n = n / b until n = 0; print remainders in reverse

Algorithm

1. Start. Read a decimal number n.
2. printBinary: while n > 0, store n mod 2 and set n = n/2; print stored values in reverse (n = 0 → print 0).
3. printOctal: same method with base 8.
4. printHexadecimal: same with base 16; if rem < 10 store rem + '0', else store (rem − 10) + 'A'.
5. Display the three results. Stop.

C Code

```c
#include <stdio.h>

// Function for Binary Conversion (Base 2)
void printBinary(int n) {
  if (n == 0) { printf("0"); return; }
  int binaryNum[32];
  int i = 0;
  while (n > 0) {
    binaryNum[i] = n % 2;
    n = n / 2;
    i++;
  }
  for (int j = i - 1; j >= 0; j--) {
    printf("%d", binaryNum[j]);
  }
}

// Function for Octal Conversion (Base 8)
void printOctal(int n) {
  if (n == 0) { printf("0"); return; }
  int octalNum[32];
  int i = 0;
  while (n > 0) {
    octalNum[i] = n % 8;
    n = n / 8;
    i++;
  }
  for (int j = i - 1; j >= 0; j--) {
    printf("%d", octalNum[j]);
  }
}
```

```c
// Function for Hexadecimal Conversion (Base 16)
void printHexadecimal(int n) {
  if (n == 0) { printf("0"); return; }
  char hexNum[32];
  int i = 0;
  while (n > 0) {
    int rem = n % 16;
    if (rem < 10) {
      hexNum[i] = rem + '0';
    } else {
      hexNum[i] = (rem - 10) + 'A';
    }
    n = n / 16;
    i++;
  }
  for (int j = i - 1; j >= 0; j--) {
    printf("%c", hexNum[j]);
  }
}

int main() {
  int decimal;
  printf("Enter any decimal number: ");
  scanf("%d", &decimal);
  printf("\n--- Results ---\n");
  printf("i) Binary Number: ");
  printBinary(decimal);
  printf("\n");
  printf("ii) Octal Number: ");
  printOctal(decimal);
  printf("\n");
  printf("iii) Hexadecimal Number: ");
  printHexadecimal(decimal);
  printf("\n");
  return 0;
}
```

Sample Input / Output (input is typed in blue)

#1

```
Enter any decimal number: 25

--- Results ---
i) Binary Number: 11001
ii) Octal Number: 31
iii) Hexadecimal Number: 19
```

#2

```
Enter any decimal number: 255

--- Results ---
i) Binary Number: 11111111
ii) Octal Number: 377
iii) Hexadecimal Number: FF
```

#3

```
Enter any decimal number: 2026

--- Results ---
i) Binary Number: 11111101010
ii) Octal Number: 3752
iii) Hexadecimal Number: 7EA
```

Problem 05 · First n Prime Numbers in Tabular Form

num is prime if no i with 2 ≤ i and i × i ≤ num divides it (num mod i ⁄ 0); 5 numbers per row, width %-8d

Algorithm

1. Start. Read n. If n ≤ 0, display “Invalid Input” and stop.
2. Initialize count = 0, num = 2.
3. While count < n:
   a. isPrime = true; for i = 2 while i × i ≤ num: if num mod i == 0, isPrime = false, break.
   b. If isPrime: print num (width 8), count++; if count mod 5 == 0 print newline.
   c. num++.
4. Stop.

C Code

```c
#include <stdio.h>
#include <stdbool.h>

// Function to check if a number is prime
bool checkPrime(int num) {
  if (num < 2) return false;
  for (int i = 2; i * i <= num; i++) {
    if (num % i == 0) {
      return false;
    }
  }
  return true;
}

int main() {
  int n;
  printf("Enter the value of n (number of prime numbers): ");
  scanf("%d", &n);
  if (n <= 0) {
    printf("Please enter a positive integer greater than 0.\n");
    return 0;
  }
  printf("\n=========================================\n");
  printf("        FIRST %d PRIME NUMBERS           \n", n);
  printf("=========================================\n\n");
  int count = 0;
  int num = 2;
  // Loop until we find n prime numbers
  while (count < n) {
    if (checkPrime(num)) {
      printf("%-8d", num); // Tabular formatting with width 8
      count++;
      // Print 5 prime numbers per line
      if (count % 5 == 0) {
        printf("\n");
      }
    }
    num++;
  }
  printf("\n");
  return 0;
```

```
}
```

Sample Input / Output (input is typed in blue)

```
#1 Enter the value of n (number of prime numbers): 5

   ========================================
           FIRST 5 PRIME NUMBERS
   ========================================

   2       3       5       7       11
```

```
#2 Enter the value of n (number of prime numbers): 12

   ========================================
           FIRST 12 PRIME NUMBERS
   ========================================

   2       3       5       7       11
   13      17      19      23      29
   31      37
```

```
#3 Enter the value of n (number of prime numbers): 0
   Please enter a positive integer greater than 0.
```

Problem 06 · First n Fibonacci Numbers

$F_i = F(i-1) + F(i-2)\ (i \geq 2),\ F_0 = 0,\ F_1 = 1$

Algorithm

1. Start. Read n. If n ≤ 0, display error and stop.
2. Initialize a = 0 and b = 1.
3. Loop i = 1 to n: print a; nextTerm = a + b; a = b; b = nextTerm.
4. Stop.

C Code

```c
#include <stdio.h>

int main() {
  int n;
  long long a = 0, b = 1, nextTerm;
  printf("Enter the number of terms (n): ");
  scanf("%d", &n);
  if (n <= 0) {
    printf("Please enter a positive integer greater than 0.\n");
    return 0;
  }
  printf("\nThe first %d Fibonacci numbers are:\n", n);
  for (int i = 1; i <= n; i++) {
    printf("%lld ", a);
    nextTerm = a + b; // Calculate the next term
    a = b;            // Shift 'a' to 'b'
    b = nextTerm;     // Shift 'b' to 'nextTerm'
  }
  printf("\n");
  return 0;
}
```

Sample Input / Output (input is typed in blue)

```
#1 Enter the number of terms (n): 10

   The first 10 Fibonacci numbers are:
   0 1 1 2 3 5 8 13 21 34
```

```
#2 Enter the number of terms (n): 20

   The first 20 Fibonacci numbers are:
   0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
```

```
#3 Enter the number of terms (n): 0
   Please enter a positive integer greater than 0.
```

Problem 07 · Sorting an Array in Descending Order (Bubble Sort)

Adjacent pair j, j+1: if arr[j] < arr[j+1] then swap (temp = arr[j]; arr[j] = arr[j+1]; arr[j+1] = temp); n − 1 passes

Algorithm

1. Start. Read the size n, then read n integers into arr.

2. For i = 0 to n − 2:

For j = 0 to n − i − 2: if arr[j] < arr[j+1], swap them using temp.

3. Display the sorted array. Stop.

C Code

```c
#include <stdio.h>

int main() {
  int n, temp;
  // Input size of array
  printf("Enter the number of elements: ");
  scanf("%d", &n);
  int arr[n];
  // Input array elements
  printf("Enter %d integers:\n", n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
  }
  // Bubble Sort in Descending Order
  for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
      // Swap if current element is smaller than the next element
      if (arr[j] < arr[j + 1]) {
        temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
      }
    }
  }
  // Display sorted array
  printf("\nSorted array in descending order:\n");
  for (int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
  }
  printf("\n");
```

```
  return 0;
}
```

Sample Input / Output (input is typed in blue)

```
#1 Enter the number of elements: 5
   Enter 5 integers:
   12 45 7 23 19

   Sorted array in descending order:
   45 23 19 12 7
```

```
#2 Enter the number of elements: 6
   Enter 6 integers:
   3 9 3 1 9 5

   Sorted array in descending order:
   9 9 5 3 3 1
```

```
#3 Enter the number of elements: 4
   Enter 4 integers:
   -2 8 0 -7

   Sorted array in descending order:
   8 0 -2 -7
```

Problem 08 · Addition and Multiplication of Two Square Matrices (Functions)

S(ij) = A(ij) + B(ij); P(ij) = Σ A(ik) · B(kj) (k = 0 … N−1) [element prompts are shown row-wise in samples]

Algorithm

1. Start. Read the order N; call readMatrix for A and B.
2. addMatrices: for i, j = 0 to N − 1: sum[i][j] = A[i][j] + B[i][j].
3. multiplyMatrices: for i, j = 0 to N − 1: prod[i][j] = 0; for k = 0 to N − 1: prod[i][j] += A[i][k] × B[k][j].
4. printMatrix: display A, B, sum and prod. Stop.

C Code

```c
#include <stdio.h>
#define MAX 10

// Function to read a matrix
void readMatrix(int mat[MAX][MAX], int n, char name) {
  printf("\nEnter elements for Matrix %c (%dx%d):\n", name, n, n);
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      printf("%c[%d][%d] = ", name, i, j);
      scanf("%d", &mat[i][j]);
    }
  }
}

// Function to print a matrix
void printMatrix(int mat[MAX][MAX], int n) {
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      printf("%d\t", mat[i][j]);
    }
    printf("\n");
```

```c
  }
}

// Function to add two square matrices
void addMatrices(int A[MAX][MAX], int B[MAX][MAX],
                 int sum[MAX][MAX], int n) {
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      sum[i][j] = A[i][j] + B[i][j];
    }
  }
}

// Function to multiply two square matrices
void multiplyMatrices(int A[MAX][MAX], int B[MAX][MAX],
                      int prod[MAX][MAX], int n) {
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      prod[i][j] = 0;
      for (int k = 0; k < n; k++) {
        prod[i][j] += A[i][k] * B[k][j];
      }
    }
  }
}

int main() {
  int n;
  int A[MAX][MAX], B[MAX][MAX], sum[MAX][MAX], prod[MAX][MAX];
  printf("Enter the order of square matrix (N x N): ");
  scanf("%d", &n);
  // Reading matrices
  readMatrix(A, n, 'A');
  readMatrix(B, n, 'B');
  // Performing matrix operations
  addMatrices(A, B, sum, n);
  multiplyMatrices(A, B, prod, n);
  // Displaying entered matrices and results
  printf("\n--- Entered Matrix A ---\n");
  printMatrix(A, n);
  printf("\n--- Entered Matrix B ---\n");
  printMatrix(B, n);
  printf("\n--- Resultant Matrix (Addition: A + B) ---\n");
  printMatrix(sum, n);
  printf("\n--- Resultant Matrix (Multiplication: A * B) ---\n");
  printMatrix(prod, n);
  return 0;
}
```

Sample Input / Output (input is typed in blue)

#1

```
Enter the order of square matrix (N x N): 2

Enter elements for Matrix A (2x2):
A[0][0] = 1  A[0][1] = 2
A[1][0] = 3  A[1][1] = 4

Enter elements for Matrix B (2x2):
B[0][0] = 5  B[0][1] = 6
B[1][0] = 7  B[1][1] = 8

--- Entered Matrix A ---
1       2
3       4

--- Entered Matrix B ---
5       6
7       8

--- Resultant Matrix (Addition: A + B) ---
6       8
10      12

--- Resultant Matrix (Multiplication: A * B) ---
19      22
43      50
```

#2

```
Enter the order of square matrix (N x N): 2

Enter elements for Matrix A (2x2):
A[0][0] = 2  A[0][1] = 0
A[1][0] = 1  A[1][1] = 3

Enter elements for Matrix B (2x2):
B[0][0] = 1  B[0][1] = 4
B[1][0] = 0  B[1][1] = 2

--- Entered Matrix A ---
2       0
1       3

--- Entered Matrix B ---
1       4
0       2

--- Resultant Matrix (Addition: A + B) ---
3       4
1       5

--- Resultant Matrix (Multiplication: A * B) ---
2       8
1       10
```

```
#3 Enter the order of square matrix (N x N): 3

   Enter elements for Matrix A (3x3):
   A[0][0] = 1  A[0][1] = 2  A[0][2] = 3
   A[1][0] = 4  A[1][1] = 5  A[1][2] = 6
   A[2][0] = 7  A[2][1] = 8  A[2][2] = 9

   Enter elements for Matrix B (3x3):
   B[0][0] = 9  B[0][1] = 8  B[0][2] = 7
   B[1][0] = 6  B[1][1] = 5  B[1][2] = 4
   B[2][0] = 3  B[2][1] = 2  B[2][2] = 1

   --- Entered Matrix A ---
   1       2       3
   4       5       6
   7       8       9

   --- Entered Matrix B ---
   9       8       7
   6       5       4
   3       2       1

   --- Resultant Matrix (Addition: A + B) ---
   10      10      10
   10      10      10
   10      10      10

   --- Resultant Matrix (Multiplication: A * B) ---
   30      24      18
   84      69      54
   138     114     90
```

Problem 09 · Frequency of Every Character in a Line of Text

freq[256] indexed by ASCII code: freq[c] = freq[c] + 1 for each character; fgets() reads spaces too

Algorithm

1. Start. Declare str[1000] and freq[256] = {0}. Read a line using fgets().
2. For each character of str (except '\n'): increment freq[(unsigned char)str[i]].
3. Traverse str again: if ch != '\n' and freq[ch] > 0, print ch (or 'Space') with freq[ch], then set freq[ch] = 0.
4. Stop.

C Code

```c
#include <stdio.h>
#include <string.h>

int main() {
  char str[1000];
  int freq[256] = {0}; // Initialize all frequencies to 0
  // Input line of text
  printf("Enter a line of text: ");
  fgets(str, sizeof(str), stdin);
  // Step 1: Count frequency of each character
  for (int i = 0; str[i] != '\0'; i++) {
    if (str[i] != '\n') { // Ignore trailing newline from fgets
```

```
      freq[(unsigned char)str[i]]++;
    }
  }
  printf("\n--- Character Frequencies ---\n");
  // Step 2: Display frequencies in original order of appearance
  for (int i = 0; str[i] != '\0'; i++) {
    unsigned char ch = str[i];
    if (ch != '\n' && freq[ch] > 0) {
      if (ch == ' ') {
        printf("'Space' = %d\n", freq[ch]);
      } else {
        printf("'%c' = %d\n", ch, freq[ch]);
      }
      freq[ch] = 0; // Prevent printing duplicate output
    }
  }
  return 0;
}
```

Sample Input / Output (input is typed in blue)

#1

```
Enter a line of text: hello world

--- Character Frequencies ---
'h' = 1
'e' = 1
'l' = 3
'o' = 2
'Space' = 1
'w' = 1
'r' = 1
'd' = 1
```

#2

```
Enter a line of text: banana

--- Character Frequencies ---
'b' = 1
'a' = 3
'n' = 2
```

#3

```
Enter a line of text: PUST ICE 2026

--- Character Frequencies ---
'P' = 1
'U' = 1
'S' = 1
'T' = 1
'Space' = 2
'I' = 1
'C' = 1
'E' = 1
'2' = 2
'0' = 1
'6' = 1
```

Problem 10 · sin(x) Using Its Infinite Series (Two User-Defined Functions)

$\sin x = x - x^3/3! + x^5/5! - \ldots$; $T_1 = x$, $T(k) = -T(k-1) \cdot x^2 / ((2k-2)(2k-1))$; $x(\text{rad}) = x(\text{deg}) \times \pi/180$

Algorithm

1. Start. Read angle x_deg (degrees) and number of terms n.
2. x_rad = degToRad(x_deg) = x_deg × (π / 180).
3. calculateSine(x_rad, n): sum = term = x_rad; for i = 2 to n: term = −term × x_rad² / ((2i − 2)(2i − 1)); sum = sum + term; return sum.
4. Display the calculated sine value (and built-in sin for comparison). Stop.

C Code

```c
#include <stdio.h>
#include <math.h>
#define PI 3.14159265358979323846

// Function 1: Converts degree to radian
double degToRad(double degree) {
  return degree * (PI / 180.0);
}

// Function 2: Calculates sine series sum up to n terms
double calculateSine(double x_rad, int n) {
  double sum = x_rad;
  double term = x_rad;
  for (int i = 2; i <= n; i++) {
    // Compute current term from previous term
    term = -term * (x_rad * x_rad) / ((2 * i - 2) * (2 * i - 1));
    sum += term;
  }
  return sum;
}

int main() {
  double degree, x_rad, sin_val;
  int n;
  printf("Enter angle in degrees (x): ");
  scanf("%lf", &degree);
  printf("Enter number of terms (n): ");
  scanf("%d", &n);
  // Convert degree to radian using Function 1
  x_rad = degToRad(degree);
  // Calculate sine using Function 2
  sin_val = calculateSine(x_rad, n);
  printf("\n--- Results ---\n");
  printf("Angle in radians: %.6lf rad\n", x_rad);
  printf("Calculated sin(%.2lf) using %d terms = %.6lf\n",
         degree, n, sin_val);
  printf("Built-in sin(%.2lf) = %.6lf\n", degree, sin(x_rad));
  return 0;
}
```

Sample Input / Output (input is typed in blue)

```
#1 Enter angle in degrees (x): 30
   Enter number of terms (n): 5

   --- Results ---
   Angle in radians: 0.523599 rad
   Calculated sin(30.00) using 5 terms = 0.500000
   Built-in sin(30.00) = 0.500000
```

```
#2 Enter angle in degrees (x): 60
   Enter number of terms (n): 6

   --- Results ---
   Angle in radians: 1.047198 rad
   Calculated sin(60.00) using 6 terms = 0.866025
   Built-in sin(60.00) = 0.866025
```

```
#3 Enter angle in degrees (x): 90
   Enter number of terms (n): 7

   --- Results ---
   Angle in radians: 1.570796 rad
   Calculated sin(90.00) using 7 terms = 1.000000
   Built-in sin(90.00) = 1.000000
```

Problem 11 · Alphabetical Sorting of Strings and Saving to a File

strcmp(s1, s2) > 0 → s1 comes after s2 → swap with strcpy() and temp; fopen(name, "w"), fprintf(file, ...), fclose(file)

Algorithm

1. Start. Read n, then read n strings into str[n][100].
2. Bubble sort: for i = 0 to n − 2, for j = 0 to n − i − 2: if strcmp(str[j], str[j+1]) > 0, swap using strcpy and temp.
3. Open “sorted_strings.txt” in write mode; if file == NULL, print error and stop.
4. For i = 0 to n − 1: print str[i] on screen and write it to the file with fprintf().
5. fclose(file). Stop.

C Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
  int n;
  // Input number of strings
  printf("Enter the number of strings: ");
  scanf("%d", &n);
  char str[n][100], temp[100];
  // Input strings
  printf("Enter %d strings:\n", n);
  for (int i = 0; i < n; i++) {
    scanf("%s", str[i]);
  }
  // Sort strings in alphabetical order
  for (int i = 0; i < n - 1; i++) {
```

```c
    for (int j = 0; j < n - i - 1; j++) {
      if (strcmp(str[j], str[j + 1]) > 0) {
        strcpy(temp, str[j]);
        strcpy(str[j], str[j + 1]);
        strcpy(str[j + 1], temp);
      }
    }
  }
  // Open file to store sorted strings
  FILE *file = fopen("sorted_strings.txt", "w");
  if (file == NULL) {
    printf("Error opening file for writing!\n");
    return 1;
  }
  printf("\n--- Sorted Strings (Alphabetically) ---\n");
  for (int i = 0; i < n; i++) {
    printf("%s\n", str[i]);        // Display on screen
    fprintf(file, "%s\n", str[i]); // Write to file
  }
  // Close the file
  fclose(file);
  return 0;
}
```

Sample Input / Output (input is typed in blue)

#1

```
Enter the number of strings: 5
Enter 5 strings:
banana apple mango cherry grape

--- Sorted Strings (Alphabetically) ---
apple
banana
cherry
grape
mango
```

#2

```
Enter the number of strings: 4
Enter 4 strings:
rahim karim abdul sumon

--- Sorted Strings (Alphabetically) ---
abdul
karim
rahim
sumon
```

#3

```
Enter the number of strings: 3
Enter 3 strings:
Delta Alpha Charlie

--- Sorted Strings (Alphabetically) ---
Alpha
Charlie
Delta
```